

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

A Salesforce Based Solution

A PROJECT DOCUMENTATION

Submitted by

PRADEEP KUMAR R

NM ID: CA86DF453063BF99FCC40A1FFDE7E78F

Bachelor of Technology
in
INFORMATION TECHNOLOGY
(Final Year)

AVC COLLEGE OF ENGINEERING

Mayiladuthurai, Tamil Nadu

Academic Year: 2026 – 2027

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and the Principal of **AVC College of Engineering, Mayiladuthurai** for providing the infrastructure, laboratory facilities and academic environment needed to complete this project successfully.

I am thankful to the Head of the Department of Information Technology for the constant encouragement and the support extended throughout my final year. I also thank my project guide and all the faculty members of the department for their valuable suggestions, timely reviews and motivation during the course of the work.

I am grateful to the Naan Mudhalvan programme and the Salesforce learning platform for the structured milestones, the hands-on learning path and the Developer Edition environment that allowed me to build, test and refine this solution end to end.

Finally, I thank my parents, friends and teammates for their continuous support, patience and belief in me.

Pradeep Kumar R

ABSTRACT

Insurance carriers that offer vehicle, property and life policies often depend on legacy systems where quoting, policy issuance and claims handling are manual, slow and inconsistent. The absence of a single platform also makes cross-selling difficult and gives the customer a poor experience when a claim is raised. This project, the **Multi-Line Insurance Policy and Claims Management System**, addresses these problems with a unified Salesforce solution.

A flexible data model built from the custom objects **Policy** and **Claim**, differentiated by Record Types and Field Sets, stores the very different data required by Auto (VIN, model year), Property (square footage, year built) and Life (beneficiary name, policy term) insurance in one structure. A multi-screen **Screen Flow (AutoQuotingFlow)** guides agents through quoting, while an **Apex invocable class (PremiumCalculator)** rates the policy and writes the premium back to the draft record. A **Record-Triggered Flow** routes every new claim to the correct queue based on policy type.

For adjusters, an **Apex controller and two Lightning Web Components (claimsDashboardLwc and claimTileLwc)** provide a focused dashboard with client-side filtering. High value claims above USD 50,000 are automatically submitted to a two-step **Approval Process** (Senior Adjuster, then Department Manager), and approvers can act from a **Screen Flow quick action** placed on the Claim record. Sharing Rules based on policy state, three Permission Sets (Agent, Adjuster, Manager) and an Apex test class complete the security and quality layers.

The result is a standardised, automated and secure platform that shortens quote-to-policy time, reduces manual claim triage and gives users a 360-degree view of Customer, Policy and Claim data.

Keywords: Salesforce, Screen Flow, Record-Triggered Flow, Apex, Lightning Web Components, Approval Process, Insurance, Claims Management.

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	2
ABSTRACT.....	3
TABLE OF CONTENTS.....	4
CHAPTER 1: INTRODUCTION.....	6
1.1 Overview.....	6
1.2 Background.....	6
1.3 Problem Statement.....	6
1.4 Objectives.....	6
1.5 Scope of the Project.....	7
1.6 Challenges.....	7
1.7 Organisation of the Report.....	7
CHAPTER 2: SYSTEM ANALYSIS.....	8
2.1 Existing System and Its Limitations.....	8
2.2 Proposed System.....	8
2.3 Feasibility Study.....	8
2.4 Functional Requirements.....	8
2.5 Non-Functional Requirements.....	9
2.6 Technology Stack.....	9
2.7 User Roles.....	10
CHAPTER 3: SYSTEM DESIGN.....	11
3.1 Architecture Overview.....	11
3.2 Data Model.....	11
3.3 Record Types.....	12
3.4 Field Sets.....	12
3.5 Entity Relationships.....	12
3.6 Automation Design.....	12
3.7 Security Design.....	13
CHAPTER 4: MILESTONE 1: CORE DATA MODEL AND POLICY CONFIGURATION.....	14
4.1 Creating the Policy Custom Object.....	14
4.2 Defining Record Types on the Policy Object.....	14
4.3 Creating Field Sets for Policy Specific Data.....	14
4.4 Creating Fields on the Policy Object.....	14
4.5 Creating the Claim Object.....	15
4.6 AutoQuoting Screen Flow.....	15
4.7 Validation Rule on the Policy Object.....	16
CHAPTER 5: MILESTONE 2: COMPLEX POLICY ISSUANCE AND CLAIM ROUTING AUTOMATION.....	17
5.1 PremiumCalculator Apex Class.....	17

5.2 Integrating the Apex Action into AutoQuotingFlow.....	18
5.3 Record-Triggered Flow for Claim Routing.....	18
CHAPTER 6: MILESTONE 3: CLAIMS ADJUSTER LWC DASHBOARD DEVELOPMENT.....	20
6.1 ClaimsAdjusterController Apex Class.....	20
6.2 claimsDashboardLwc Component.....	21
6.3 claimTileLwc Reusable Component.....	23
6.4 Component Interaction.....	24
CHAPTER 7: MILESTONE 4: ADVANCED CLAIM PROCESSING, SECURITY AND CODE SETUP.....	25
7.1 Approval Status Field.....	25
7.2 Approval Process: High Value Claim Approval.....	25
7.3 Submission Automation Flow.....	26
7.4 Claim Approver Screen Flow.....	26
7.5 Apex Test Class for ClaimsAdjusterController.....	27
7.6 State Based Sharing for Adjusters.....	28
7.7 Permission Sets.....	29
CHAPTER 8: TESTING AND VALIDATION.....	30
8.1 Testing Strategy.....	30
8.2 Test Cases.....	30
8.3 Apex Unit Test Results.....	31
8.4 Observations and Known Limitations.....	31
CHAPTER 9: RESULTS AND DISCUSSION.....	31
9.1 Outcomes.....	31
9.2 Before and After Comparison.....	31
9.3 Learning Outcomes.....	32
CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS.....	32
10.1 Conclusion.....	32
10.2 Future Enhancements.....	32
REFERENCES.....	32

CHAPTER 1: INTRODUCTION

1.1 Overview

The insurance industry runs on trust, speed and accurate information. A customer who buys a policy expects a quick quote, a clear premium and a smooth process when something goes wrong and a claim must be filed. For a carrier that sells vehicle, property and life insurance, delivering this experience consistently is difficult when every line of business has its own data, its own forms and its own manual steps.

This project implements a **Multi-Line Insurance Policy and Claims Management System** on the Salesforce Platform. It combines declarative tools (custom objects, Record Types, Screen Flows, Record-Triggered Flows, Approval Processes, Sharing Rules and Permission Sets) with programmatic tools (Apex classes, Apex test classes and Lightning Web Components) to deliver a single application for agents, adjusters and managers.

1.2 Background

The carrier in this case study currently uses a legacy system. Agents re-enter customer data for each product line, quotes are produced by hand, and claims are passed between departments by email and spreadsheets. As a result, resolution times are long, operational cost is high and managers cannot see the full picture of a customer who holds more than one kind of policy.

Salesforce is a cloud CRM platform that provides a shared data layer, a workflow engine (Flow), a programming layer (Apex and Lightning Web Components) and a mature security model. Because these capabilities are available on one platform, the carrier can replace several disconnected tools with one application and one source of truth.

1.3 Problem Statement

An insurance carrier, which manages vehicle, property and life insurance, uses a legacy system that results in manual, slow and inconsistent policy quoting, issuance and claims handling. The lack of a unified platform makes cross-selling difficult, and the manual claims review process leads to long resolution times, high operational costs and poor customer experience. The business needs a single Salesforce solution to:

- Standardize and accelerate the quoting and policy issuance process for all insurance lines.
- Automate claim initiation and routing based on policy type and claim severity.
- Provide a 360-degree view of the Customer, Policy and Claim data in one place.

1.4 Objectives

- Design a flexible data model capable of storing diverse product data, for example vehicle VIN for Vehicle and square footage for Property.
- Implement Screen Flows that guide agents through complex, multi-step quoting processes for different policy types.
- Develop a dynamic Apex and LWC Claims Dashboard so adjusters can manage their workload efficiently.
- Automate claims approval and escalation based on claim amount and policy history using Record-Triggered Flows.

- Enforce data validation across all insurance products to ensure compliance and accuracy.

1.5 Scope of the Project

The scope covers four milestones: (1) the core data model and policy configuration, (2) policy issuance with premium calculation and claim routing automation, (3) the Claims Adjuster dashboard built with Apex and LWC, and (4) advanced claim processing with approvals, test coverage, sharing and permission sets. The Auto line is implemented end to end in the quoting flow; the Property and Life lines are supported by the same data model, Record Types and Field Sets and can be extended with their own quoting flows using the same pattern.

The project is built and demonstrated in a Salesforce Developer Edition org. A real deployment would additionally require integration with the carrier's payment, document and external data providers, which are outside the scope of this work.

1.6 Challenges

Variable data management. Vehicle, Property and Life policies need different fields. Creating a separate object for each line would fragment reporting, so the solution uses one Policy object with Record Types and Field Sets to present only the relevant fields for each line.

Approval complexity. Claims must be routed based on policy type, claim amount and the level of the adjuster. This is handled by combining a Record-Triggered Flow for routing with a sequential, two-step Approval Process for high value claims.

API integration simulation. The business wants quotes verified against an external motor vehicle or property database. Such a callout is simulated in Apex and must be accompanied by mock test classes so that tests can run without a real endpoint.

User adoption. Agents are measured on speed. The Screen Flows are therefore designed to minimise clicks, group related inputs on a single screen and pre-fill values wherever possible.

1.7 Organisation of the Report

- **Chapter 2** analyses the existing system, the proposed system, requirements and technology stack.
- **Chapter 3** presents the architecture, data model and security design.
- **Chapters 4 to 7** describe the implementation of Milestones 1 to 4 step by step.
- **Chapter 8** documents testing and observations.
- **Chapters 9 and 10** discuss results, conclusions and future enhancements.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System and Its Limitations

In the existing arrangement, each insurance line is handled by a separate legacy module or even by separate spreadsheets. The following limitations were identified:

- Quotes are prepared manually, so two agents may quote different premiums for similar risks.
- Policy data is stored in line-specific formats; there is no common customer view and cross-selling opportunities are missed.
- Claims are registered on paper or by email and are assigned to adjusters manually, causing delays.
- There is no automatic escalation for high value claims; approvals depend on follow-ups.
- Validation is weak, which allows incomplete or invalid data such as a malformed VIN to enter the system.
- Access control is coarse, so adjusters may see claims that are not relevant to them.

2.2 Proposed System

The proposed system replaces the legacy tools with one Salesforce application. Policies and claims are stored in custom objects that share a common structure and are specialised by Record Types. Agents create quotes through a guided Screen Flow; the premium is calculated by Apex and stored on the policy. When a claim is created, a Record-Triggered Flow identifies the policy type and assigns the claim to the right queue. Adjusters work from a Lightning dashboard that shows only their own claims, and claims above USD 50,000 enter an automatic approval chain.

2.3 Feasibility Study

Technical feasibility. All required capabilities (custom objects, Flow, Apex, LWC, Approval Processes, Sharing Rules and Permission Sets) are standard features of the Salesforce Platform, and the Developer Edition org used for development is free of cost.

Economic feasibility. Because the solution is configured on an existing platform instead of being built from scratch, development effort and infrastructure cost are low. Reduced manual effort and faster claim resolution are expected to offset the licence cost.

Operational feasibility. Screen Flows and a focused dashboard mirror the way agents and adjusters already work, so training effort is small. Role based Permission Sets give each user only the access they need.

2.4 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Delivered by
FR-1	Store Auto, Property and Life policies in one object with line-specific fields	Policy object, Record Types, Field Sets
FR-2	Guide agents through a multi-step quoting process	AutoQuotingFlow (Screen Flow)

ID	Requirement	Delivered by
FR-3	Calculate the premium for a draft policy	PremiumCalculator (Apex invocable)
FR-4	Validate VIN length for Auto policies	Validation rule VIN_Must_Be_17_Characters
FR-5	Route each new claim to a queue by policy type	Record-Triggered Flow on Claim
FR-6	Show adjusters their assigned claims with filtering	ClaimsAdjusterController, claimsDashboardLwc, claimTileLwc
FR-7	Escalate claims above USD 50,000 for two-level approval	Approval Process and Submission Automation Flow
FR-8	Allow approvers to approve or reject from the record	Claim Approver Screen Flow and quick action
FR-9	Restrict claim visibility by policy state	Sharing Rules and state update flow
FR-10	Separate access for Agents, Adjusters and Managers	Three Permission Sets

2.5 Non-Functional Requirements

- **Performance:** the dashboard uses a cacheable Apex method and client-side filtering so that filtering does not require a server round trip.
- **Security:** object permissions, sharing rules and Apex class access are granted only through Permission Sets.
- **Usability:** quoting is completed in two screens and one confirmation, with pick-lists and lookups instead of free text wherever possible.
- **Maintainability:** logic is split into small flows and focused Apex classes; automatic numbering (P-0001, C-0001) gives readable record names.
- **Quality:** Apex code is accompanied by a test class targeting more than 95 percent coverage.

2.6 Technology Stack

Table 2.2: Tools and technologies

Layer	Technology	Purpose
Data	Custom objects, Record Types, Field Sets	Unified storage of Policy and Claim data
Declarative automation	Screen Flow, Record-Triggered Flow	Quoting, routing, approval submission
Approvals	Approval Process	Two-step approval for high value claims
Programmatic logic	Apex (invocable method, controller, test class)	Rating logic and data retrieval
User interface	Lightning Web Components, SLDS	Claims dashboard and claim tiles

Layer	Technology	Purpose
Security	Sharing Rules, Permission Sets	Record and object level access
Tooling	Developer Console, Lightning Studio extension	Code authoring and deployment

2.7 User Roles

Insurance Agents create quotes and draft policies and read claims for customer service. **Claims Adjusters** work assigned claims from the dashboard and edit claim details. **Senior Adjusters** are first level approvers for high value claims. **Department Managers** are second level approvers who also run reports and manage approvals.

CHAPTER 3: SYSTEM DESIGN

3.1 Architecture Overview

The solution is organised in five cooperating layers. The **data layer** holds the Policy and Claim objects and their Record Types. The **automation layer** contains the Screen Flows and Record-Triggered Flows. The **logic layer** contains Apex classes for rating and data retrieval. The **presentation layer** is the Lightning dashboard built from two LWCs. The **security layer** applies sharing rules and Permission Sets across all of the above.

3.2 Data Model

Two custom objects form the core of the data model. **Policy__c** represents a contract between the carrier and a customer (a Contact). **Claim__c** represents a loss event raised against a policy. Both use auto-number record names so that every record has a short, unique and readable identifier.

Table 3.1: Policy object fields

Field label	Data type	Used for
Policy Number (Name)	Auto Number, P-{0000}	Unique policy identifier
Customer	Lookup(Contact)	Policy holder
Policy Start Date	Date	Effective date
Policy Term Months	Number(18,0)	Life policy duration
Premium	Currency(16,2)	Output of PremiumCalculator
VIN	Text(20)	Auto: vehicle identification number
Model Year	Text(5)	Auto: year of manufacture
Square Footage	Number(18,0)	Property: built-up area
Year Built	Text(5)	Property: construction year
Beneficiary Name	Text(50)	Life: nominee
Policy State	Picklist	Rating by state; used by the quoting flow

Table 3.2: Claim object fields

Field label	Data type	Used for
Claim Number (Name)	Auto Number, C-{0000}	Unique claim identifier
Policy	Lookup(Policy)	Parent policy
Adjuster	Lookup(User)	Responsible adjuster
Claim Amount	Currency(16,2)	Severity and approval threshold
Approval Status	Picklist: New, Submitted for Approval, Approved, Rejected	Approval tracking
Date of Loss	Date/Time	Age of the claim
Description	Long Text Area	Narrative of the incident

Field label	Data type	Used for
Policy Account Holder State	Text / Picklist	Criteria for state based sharing rules

3.3 Record Types

Table 3.3: Record Types

Object	Record Types	Purpose
Policy	Auto, Property, Life	Controls which line of business a policy belongs to and which field set is shown
Claim	Accident, Property, Life	Classifies the nature of the claim

3.4 Field Sets

Field Sets group the fields that belong to one product line. They allow the same component or page to render only the Vehicle, Property or Life fields without hard-coding field names.

Table 3.4: Field Sets on the Policy object

Field Set	Fields
Vehicle	VIN, Model Year
Property	Square Footage, Year Built
Life	Beneficiary Name, Policy Term Months

3.5 Entity Relationships

A Contact (customer) can hold many Policies, and a Policy can have many Claims. Each Claim is also associated with an Adjuster through a User lookup and is owned by a queue or user, which drives the dashboard. This structure gives a 360-degree view: from a Contact, a user can navigate to every Policy and from every Policy to all Claims.

3.6 Automation Design

Table 3.5: Automation inventory

Name	Type	Trigger	Result
AutoQuotingFlow	Screen Flow	Agent starts quote	Draft policy created and premium stored
Claim routing flow	Record-Triggered (after save)	Claim created	Owner set to Auto, Property or Life queue
Submission Automation Flow	Record-Triggered (after save)	Claim amount above 50,000 and status New	Claim submitted to approval
Claim Approver Screen Flow	Screen Flow	Approver clicks quick action	Approve or reject with comments

Name	Type	Trigger	Result
Claim Policy Holder State Update	Record-Triggered (after save)	Claim created or updated	State copied from policy to claim

3.7 Security Design

Security follows the principle of least privilege. Object permissions are granted through three Permission Sets: Insurance Agent Access (quoting only), Claims Adjuster Access (claim handling) and Claims Manager Access (reporting and approvals). Record visibility on claims is further narrowed by criteria based sharing rules on the policy holder state, and the approval process restricts editing to administrators while a claim is in approval.

CHAPTER 4: MILESTONE 1: CORE DATA MODEL AND POLICY CONFIGURATION

Milestone 1 builds the foundation: the custom objects, their Record Types and fields, the AutoQuoting Screen Flow and the data validation rule.

4.1 Creating the Policy Custom Object

1. Go to **Setup** and open **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter Label **Policy**, Plural Label **Policies**; the Object Name is auto populated (Policy__c).
4. Set Record Name to **Policy Number**, Data Type **Auto Number**, Display Format **P-{0000}** and Starting Number **1**.
5. Under Optional Features select **Allow Reports**, **Allow Activities** and **Track Field History**.
6. Set Deployment Status to **Deployed** and tick **Allow Search**.
7. Click **Save**.

4.2 Defining Record Types on the Policy Object

1. In Object Manager, open **Policy__c** and select **Record Types** from the left panel.
2. Click **New**, choose the existing record type **Master**, and enter the label **Auto**.
3. The Record Type Name is auto populated. Add a meaningful description and tick **Active**.
4. Tick **Make Available** for all profiles and click **Next**.
5. On the page layout assignment screen leave the defaults and click **Save**.
6. Repeat the same steps to create the **Property** and **Life** Record Types.

4.3 Creating Field Sets for Policy Specific Data

1. Open **Policy__c** and select **Field Sets**, then click **New**.
2. Enter the label **Vehicle**; the name is auto populated. Keep "Where is this used" as the Policy object and click **Save**.
3. Open the new field set and drag the **VIN** and **Model Year** fields into it, then **Save**.
4. Repeat to create the **Property** field set (Square Footage, Year Built) and the **Life** field set (Beneficiary Name, Policy Term Months).

4.4 Creating Fields on the Policy Object

The fields listed in Table 3.2 are created one by one from **Fields and Relationships**. Customer is a Lookup to Contact, Premium is Currency (16,2), Policy Start Date is a Date, Policy Term Months and Square Footage are Numbers (18,0), and Beneficiary Name (50), Model Year (5), VIN (20) and Year Built (5) are Text fields. The Policy State picklist is added to support the quoting flow and the rating logic.

4.5 Creating the Claim Object

1. In Object Manager choose **Create > Custom Object**.
2. Enter Label **Claim** and Plural Label **Claims**; set Record Name to **Claim Number**, Data Type **Auto Number**, Display Format **C-{0000}**, Starting Number **1**.
3. Select **Allow Reports**, **Allow Activities**, **Track Field History**, set status to **Deployed** and tick **Allow Search**. Click **Save**.
4. Create the Record Types **Accident**, **Property** and **Life** using the same procedure as for Policy.
5. Create the fields Adjuster (Lookup User), Approval Status (Picklist), Claim Amount (Currency 16,2), Date of Loss (Date/Time), Description (Long Text Area) and Policy (Lookup Policy).

4.6 AutoQuoting Screen Flow

The AutoQuotingFlow lets an agent capture an Auto policy quote in two screens and creates a draft Policy record. It is built from the following elements.

Step 1: Get Records – Get Vehicle RT ID

Immediately after the Start element, a Get Records element named **Get Vehicle RT ID** reads the Record Type object with two conditions: sObject Type Name equals Policy__c and Record Type Name equals Auto. The Id returned is later assigned to the new policy so that it is created with the Auto Record Type.

Step 2: Screen – New Auto Policy Quote – Basic Information

The first screen collects common data: a display text (Screen_1_Policy_Basics), a **Lookup** component labelled **Policy Holder** bound to Customer__c on Policy__c, a **Date** component labelled **Policy Start Date**, and a **Picklist** labelled **Policy State** that uses a Picklist Choice Set created from the Policy State field.

Step 3: Screen – Vehicle-Specific Details

The second screen (Screen_2_Vehicle_Details) shows two text inputs, **VIN** and **Model year**, that capture the data specific to a vehicle.

Step 4: Create Records – Create Draft Policy

A Create Records element stores field values manually on the Policy object, with the mapping shown in Table 4.1, and saves the new record Id in a text variable **varPolicyId**.

Table 4.1: Field mapping in Create Draft Policy

Policy field	Value from
Customer	Basic Information screen > Policy Holder > Record ID
Model Year	Vehicle screen > Model Year
Policy Start Date	Basic Information screen > Policy Start Date
Policy State	Basic Information screen > Policy State
RecordTypeId	Get Vehicle RT ID > Record Type ID
VIN	Vehicle screen > VIN

The flow is saved as **AutoQuotingFlow** and activated.

4.7 Validation Rule on the Policy Object

To keep data accurate, a validation rule named **VIN_Must_Be_17_Characters** is created on Policy__c. It fires only for Auto policies and blocks saving when the VIN is not exactly 17 characters long, which is the international standard length.

Validation rule formula

```
AND(  
  RecordType.DeveloperName = "Auto",  
  LEN( VIN__c ) <> 17  
)
```

The error message is "The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies." and the error location is set to the VIN field, so that the agent sees the message next to the field that needs correction.

CHAPTER 5: MILESTONE 2: COMPLEX POLICY ISSUANCE AND CLAIM ROUTING AUTOMATION

Milestone 2 adds business logic. An Apex invocable class calculates the premium, the quoting flow is extended to call it, and a Record-Triggered Flow assigns every new claim to the correct queue.

5.1 PremiumCalculator Apex Class

The class **PremiumCalculator** exposes an invocable method so that it can be called as an action from Flow. It accepts a list of wrapper objects, each carrying a policy Id, queries the corresponding policies in a single SOQL query (bulk safe) and returns a list of calculated premiums.

Creating the class

1. Click the **Setup** icon, right click **Developer Console** and open the link in a new tab.
2. In the Developer Console select **File > New > Apex Class**, enter the name **PremiumCalculator** and click **OK**.
3. Paste the code given below and save with **Ctrl + S**.

PremiumCalculator.cls

```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on
policy fields (VIN, Model Year, State).')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {

        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();

        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }

        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
            FROM Policy__c
            WHERE Id IN :policyIds
        ];

        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00;

            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            } else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }

            if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018)
            {
                basePremium += 100;
            }

            premiumResults.add(basePremium);
        }

        return premiumResults;
    }
}
```

```
}
}
```

Rating logic

Table 5.1: Premium rating rules

Condition	Effect	Example result
Base premium	Starts at 1000.00	1000.00
Policy State = CA	Multiply by 1.15	1150.00
Policy State = TX	Multiply by 1.05	1050.00
Model Year earlier than 2018	Add 100.00	CA + 2015 = 1250.00

The use of the **@InvocableVariable** wrapper keeps the interface clean for Flow, and the single SOQL query outside the loop keeps the class within governor limits even when many policies are rated together.

Simulated external verification

The problem statement requires a simulated callout to an external motor vehicle or property database for quote verification. In Apex this is implemented with an **HttpCalloutMock** implementation that returns a fixed JSON response, so that the verification logic and its tests can execute without a live endpoint. The same pattern is used for every line of business and keeps tests deterministic.

5.2 Integrating the Apex Action into AutoQuotingFlow

1. Open **Flows** from Setup and open **AutoQuotingFlow** in Flow Builder.
2. After the Create Records element add an **Action** element labelled **Calculate Premium Action**.
3. In Set Input Values, assign **policyId** to the variable **varPolicyId**.
4. Under output, tick "manually assign variables" and create a new variable resource **varCalculatedPremium** of data type Currency with 2 decimal places.
5. After the action add an **Update Records** element labelled **Update Policy With Premium**. Use "Specify conditions": object Policy, condition Record Id equals **varPolicyId**.
6. Set the Premium field to **varCalculatedPremium**.
7. Save as a new version and **Activate**.

5.3 Record-Triggered Flow for Claim Routing

This flow runs after a Claim is created and assigns it to the queue that matches the policy type. It uses a Get Records element, a Decision element and three Update Records elements.

1. Create a new **Record-Triggered Flow** on the **Claim** object, triggered when a record is **created**, optimised for **Actions and Related Records**.
2. Add **Get Records** named **Get_Policy_RT** on Policy with the condition Record Id equals Triggering Claim > Policy; store only the first record and store the Record Type in a text variable **varPolicyRTName**.

3. Add a **Decision** named **Route by Policy Type** with outcomes **Auto Claim Route** (varPolicyRTName equals Auto) and **Property Claim Route** (varPolicyRTName equals Property); the default outcome is **Life Claim Route**.
4. On the Auto path add an **Update Records** element **Assign to Auto Queue**: object Claim, condition Record Id equals Triggering Claim > Record ID, and set OwnerId to the 18 digit Id of the Auto queue.
5. Repeat for **Assign to Property Queue** and **Assign to Life Queue** with the respective queue Ids.
6. Save and activate the flow.

Table 5.2: Routing outcomes

Policy Record Type	Outcome	Claim owner
Auto	Auto Claim Route	Auto queue
Property	Property Claim Route	Property queue
Life (default)	Life Claim Route	Life queue

Design note. The queue Ids are hard coded as required by the milestone. In a production release these would be replaced by a custom metadata type or a Get Records lookup on the Group object by developer name, so that the flow is portable between sandboxes and production.

CHAPTER 6: MILESTONE 3: CLAIMS ADJUSTER LWC DASHBOARD DEVELOPMENT

Milestone 3 delivers the adjuster experience: an Apex controller that returns the logged-in adjuster's claims together with policy and customer information, and two Lightning Web Components that present the data.

6.1 ClaimsAdjusterController Apex Class

The controller defines a wrapper class **ClaimWrapper** with the key metrics the dashboard needs: claim Id, claim number, amount, status, policy type, policy holder name and days open. The method **getAssignedClaims** is annotated with **@AuraEnabled(cacheable=true)** so that it can be wired from LWC. It queries every Claim owned by the current user and traverses the relationships Claim > Policy > Customer (Contact) in one SOQL statement.

ClaimsAdjusterController.cls

```
public class ClaimsAdjusterController {

    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id;
            this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c;
            this.status = c.Approval_Status__c;

            Policy__c policyRecord = (Policy__c)p;

            if (policyRecord != null && policyRecord.RecordType != null) {
                this.policyType = policyRecord.RecordType.Name;
            } else {
                this.policyType = 'Unknown';
            }

            this.policyHolderName = (customerRec != null) ? customerRec.Name : '';

            this.daysOpen = (c.Date_of_Loss__c != null)
                ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
                : 0;
        }
    }

    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {

        List<ClaimWrapper> results = new List<ClaimWrapper>();

        List<Claim__c> claims = [
            SELECT
                Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
                Policy__r.Name,
                Policy__r.RecordType.DeveloperName,
                Policy__r.Customer__r.FirstName,
                Policy__r.Customer__r.LastName
            FROM Claim__c
            WHERE OwnerId = :UserInfo.getUserId()
            ORDER BY Date_of_Loss__c DESC
        ];

        for (Claim__c claim : claims) {
```

```

        Policy__c policy = claim.Policy__r;

        Contact customer = (policy != null && policy.Customer__r != null)
            ? (Contact)policy.Customer__r
            : new Contact();

        results.add(new ClaimWrapper(claim, policy, customer));
    }
    return results;
}
}

```

Key design points: the filter **OwnerId = :UserInfo.getUserId()** guarantees that an adjuster sees only their own work; the null checks protect against claims without a policy or customer; and the wrapper converts raw records into a flat structure that the UI can bind to directly.

6.2 claimsDashboardLwc Component

The parent component loads the claims through the **@wire** service, keeps the full list in **allClaims** and the displayed list in **visibleClaims**, and implements **client-side filtering** by policy type through a combobox. Because filtering happens in the browser, switching between Auto, Property and Life is instantaneous.

Creating the component

1. Open a new Chrome tab, search for the **Lightning Studio** extension, open the first result, click **Add to Chrome** and confirm **Add Extension**.
2. Open Lightning Studio from the Salesforce org, click **Create New** at the top left and choose **Lightning Web Component**.
3. Enter the name **claimsDashboardLwc**, tick **isExposed**, choose target configurations (App page, Home page, Record page) and click **Deploy**.
4. Paste the HTML, JavaScript and meta XML given below into the respective files.

claimsDashboardLwc.html

```

<template>
    <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">
        <div class="slds-m-around_medium">
            <lightning-combobox
                name="policyFilter"
                label="Filter by Policy Type"
                value="All"
                options={policyTypeOptions}
                onchange={handleFilterChange}
                class="slds-m-bottom_medium"
            ></lightning-combobox>

            <template if:true={visibleClaims}>
                <div class="slds-grid slds-wrap slds-grid_pull-padded">
                    <template for:each={visibleClaims} for:item="claim">
                        <div key={claim.claimId} class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-p-horizontal_small">
                            <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
                        </div>
                    </template>
                </div>
                <p class="slds-m-top_medium slds-text-heading_small">Total Claims Assigned: {visibleClaims.length}</p>
            </template>
            <template if:false={visibleClaims}>
                <p>No claims currently assigned to you.</p>
            </template>
        </div>
    </lightning-card>
</template>

```

```

    </lightning-card>
</template>

```

claimsDashboardLwc.js

```

import { LightningElement, wire, track } from 'lwc';
// Import the Apex method
import getAssignedClaims from '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
    // List to hold all data retrieved from Apex
    @track allClaims = [];
    // List to display after filtering
    @track visibleClaims = [];
    error;

    // Wire service to call the Apex method
    @wire(getAssignedClaims)
    wiredClaims({ error, data }) {
        if (data) {
            this.allClaims = data;
            this.visibleClaims = data; // Initially display all claims
            this.error = undefined;
        } else if (error) {
            this.error = error;
            this.allClaims = undefined;
            this.visibleClaims = undefined;
            console.error('Error retrieving claims:', JSON.stringify(error));
        }
    }

    // Client-side filtering logic (Required by M3)
    handleFilterChange(event) {
        const selectedPolicyType = event.target.value;

        if (selectedPolicyType === 'All') {
            this.visibleClaims = this.allClaims;
        } else {
            // Filter claims where policyType matches the selected value
            this.visibleClaims = this.allClaims.filter(claim =>
                claim.policyType === selectedPolicyType
            );
        }
    }

    get policyTypeOptions() {
        // Options for the filter dropdown
        return [
            { label: 'All Policy Types', value: 'All' },
            { label: 'Auto', value: 'Auto' },
            { label: 'Property', value: 'Property' },
            { label: 'Life', value: 'Life' }
        ];
    }
}

```

claimsDashboardLwc.js-meta.xml

```

<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>63.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>

```

6.3 claimTileLwc Reusable Component

The child component **claimTileLwc** receives one claim object through the **@api claimData** property and renders a card with an icon chosen by policy type, the claim number, policy holder, formatted claim amount, status and days open. Keeping the tile separate makes it reusable on other pages, for example on a Contact or Policy record page.

claimTileLwc.html

```
<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media_figure">
          <lightning-icon icon-name={policyIcon} alternative-
text={claimData.policyType} size="small"></lightning-icon>
        </div>
        <div class="slds-media_body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}
          </h2>
          <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>
        </div>
      </header>

      <div class="slds-border_top slds-p-top_small">
        <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p>

        <lightning-formatted-number
          value={claimData.claimAmount}
          format-style="currency"
          currency-code="USD"
          label="Claim Amount">
        </lightning-formatted-number>

        <p>Status: {claimData.status}</p>

        <p class="slds-text-body_regular">
          Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong>
        </p>
      </div>
    </div>
  </lightning-layout-item>
</template>
```

claimTileLwc.js

```
import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
  // @api decorator exposes this property to the parent component
  @api claimData;

  // Helper property to determine the correct icon based on Policy Type
  get policyIcon() {
    if (this.claimData && this.claimData.policyType) {
      const type = this.claimData.policyType.toLowerCase();
      if (type === 'auto') return 'utility:truck';
      if (type === 'property') return 'utility:home';
      if (type === 'life') return 'utility:connected_apps';
    }
    return 'utility:help';
  }
}
```

claimTileLwc.js-meta.xml

```
<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
```

```
<isExposed>true</isExposed>  
</LightningComponentBundle>
```

6.4 Component Interaction

When the dashboard loads, the wire adapter calls `getAssignedClaims` once and caches the result. The parent stores the data, loops over **visibleClaims** and creates one child tile per claim, passing the claim through the **claim-data** attribute. When the adjuster changes the combobox value, **handleFilterChange** replaces `visibleClaims` with a filtered copy and the template re-renders automatically, with no server call.

[Insert screenshot: Claims dashboard with tiles for all claims]

Figure 6.1: claimsDashboardLwc placed on a Lightning app page

CHAPTER 7: MILESTONE 4: ADVANCED CLAIM PROCESSING, SECURITY AND CODE SETUP

Milestone 4 completes the application with a multi-level approval process, automated submission, an approver screen flow, an Apex test class, state based sharing and role based permission sets.

7.1 Approval Status Field

1. Go to **Setup > Object Manager > Claim > Fields & Relationships** and click **New**.
2. Choose the data type **Picklist** and label it **Approval Status**.
3. Select "Enter values" and type **New**, **Submitted for Approval**, **Approved** and **Rejected** on separate lines.
4. Click **Next**, **Next** and **Save**.

7.2 Approval Process: High Value Claim Approval

The approval process routes claims above USD 50,000 first to the Senior Adjuster and then to the Department Manager using two sequential steps.

Table 7.1: Approval process configuration

Setting	Value
Process name	High Value Claim Approval
Object	Claim
Entry criteria	Claim Amount greater than 50000
Record editability	Administrators ONLY can edit records during approval
Approval page layout fields	Claim Number, Owner, Claim Amount, Adjuster, Policy
Security setting	Allow approvers to access the page only from within Salesforce
Initial submitters	Claim Owner

Approval steps

Table 7.2: Approval steps

Step	Approver	Multiple approvers	Reject behaviour
Step1 Senior Adjuster	Senior Adjuster user (auto assigned)	Based on FIRST response	Default
Step2 Manager Review	Department Manager user (auto assigned)	Based on FIRST response	Perform all rejection actions for this step AND all final rejection actions

Both steps use the criterion "all records should enter the step", which guarantees that every claim passes through both levels in sequence.

Field update actions

Table 7.3: Field updates

Action stage	Action name	Approval Status becomes
Initial Submission	Update Status Submitted	Submitted for Approval
Final Approval	Update Status Approved	Approved
Final Rejection	Update Status Rejected	Rejected

After all steps and actions are added, the approval process is **activated**.

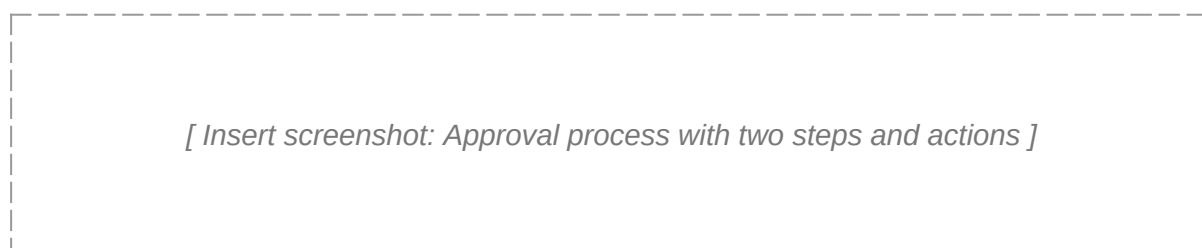


Figure 7.1: High Value Claim Approval process detail page

7.3 Submission Automation Flow

A Record-Triggered Flow submits qualifying claims automatically so that agents do not have to click the Submit for Approval button.

1. Create a **Record-Triggered Flow** on **Claim**, triggered when a record is **created or updated**.
2. Set entry conditions (all met): **Claim Amount greater than 50000** and **Approval Status equals New**.
3. For updated records select "Every time a record is updated and meets the condition requirements" and optimise for **Actions and Related Records**.
4. Add an **Action** element labelled **Submit Claim to Approval** (Submit for Approval). Set Record Id to Triggering Claim > Record ID and the approval process name to **High Value Claim Approval**.
5. Save as **Submission Automation Flow** and activate.

7.4 Claim Approver Screen Flow

Approvers should be able to review the key facts and decide without leaving the claim record. A Screen Flow is placed on the record as a quick action.

1. Create a **Screen Flow** and a variable **recordId** (Text, Available for Input).
2. Add **Get Records** named **Get Claim Details** on Claim with the condition RecordId equals recordId; store the first record and all fields.
3. Add a Screen named **Claim Review Screen**. Display the claim name, claim amount and policy type using merge fields from Get_Claim_Details.
4. Add a **Radio Buttons** component **Approval Decision** with two choice resources, **Approve** and **Reject** (text), and a **Long Text Area Approver Comments**.

5. Add a **Decision** named **Check Decision**: outcome **Decision Approved** when Approval Decision equals the Approve choice; the default outcome is **Decision Rejected**.
6. On the approved path add the **Approve Claim** action and on the rejected path the **Reject Claim** action, each with Record ID set to recordId and the approver comments passed as the submission comments.
7. Save and activate the flow as **Claim Approver Screen Flow**.

Quick action

1. In Object Manager open **Claim > Buttons, Links and Actions** and click **New Action**.
2. Choose Action Type **Flow**, select **Claim Approver Screen Flow**, label it **Approve/Reject Claim** and save.
3. Open the Claim **Page Layout**, go to **Mobile & Lightning Actions**, and drag **Approve/Reject Claim** into the Salesforce Mobile and Lightning Experience Actions section. Save.

7.5 Apex Test Class for ClaimsAdjusterController

A good Apex test sets up its own data, runs the code as a specific user and asserts the results. The test class **ClaimsAdjusterControllerTest** uses **@testSetup** to create a Standard User, a Contact, an Auto Policy and an Accident Claim all owned by that user, and then calls **getAssignedClaims** inside **System.runAs** so that the OwnerId filter returns the test claim.

ClaimsAdjusterControllerTest.cls

```
@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        // Create a real active test user
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser123434@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        // Query Policy__c RecordType
        Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
            .get('Auto')
            .getRecordTypeId();
        Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
            .get('Accident')
            .getRecordTypeId();

        // Create Contact owned by same user
        Contact con = new Contact(
            FirstName = 'John',
            LastName = 'Doe',
            OwnerId = u.Id
        );
        insert con;

        // Create Policy owned by same user
        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
```

```

        RecordTypeId = policyRT,
        OwnerId = u.Id,
        VIN__c = '1A903284375893532'
    );
    insert policy;

    // Create Claim owned by same user
    Claim__c claim = new Claim__c(
        Policy__c = policy.Id,
        RecordTypeId = claimRT,
        Claim_Amount__c = 5000,
        Date_of_Loss__c = Date.today().addDays(-10),
        OwnerId = u.Id
    );
    insert claim;
    System.debug('Claim inserted: '+claim.Id);
}
@isTest
static void testGetAssignedClaims() {
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
    System.runAs(u) {
        Test.startTest();
        List<ClaimsAdjusterController.ClaimWrapper> results =
            ClaimsAdjusterController.getAssignedClaims();
        System.debug('Result is: '+results);
        Test.stopTest();

        ClaimsAdjusterController.ClaimWrapper wrap = results[0];

        System.assertNotEquals(null, wrap.claimId);
        System.assertNotEquals(null, wrap.claimNumber);
        System.assertEquals('New', wrap.status);
        System.assertEquals(5000, wrap.claimAmount);
        System.assert(wrap.policyHolderName.contains('John'));
        System.assertNotEquals('Unknown', wrap.policyType);
    }
}
}

```

To run the test, open the class in Developer Console, click **Run Test** at the top right and open the **Tests** tab at the bottom to see the pass status. Open the **Overall Code Coverage** panel to confirm the coverage achieved by ClaimsAdjusterController.

[Insert screenshot: Developer Console Tests tab with coverage]

Figure 7.5: Apex test execution and code coverage

7.6 State Based Sharing for Adjusters

Adjusters should only see claims in their assigned territory. Because sharing rules need a field on the Claim object itself, a Record-Triggered Flow first copies the state from the policy to the claim, and criteria based sharing rules then use that field.

Flow: Claim Policy Holder State Update

1. Create a **Record-Triggered Flow** on **Claim**, triggered when a record is **created or updated**, with no entry conditions, optimised for **Actions and Related Records**.
2. Add **Get Records** **get_policy_records** on **Policy**, no filter, store all records, and keep the **Policy State** field.

3. Add a **Loop** element **Claim_Filed_update** over the collection of policies, from first item to last item.
4. Inside the loop add an **Update Records** element **Claim_Filed_state_update** that sets the **Policy Account Holder State** field on the claim from the current loop item's Policy State.
5. Save and activate as **Claim Policy Holder State Update**.

Sharing rules

1. Open **Setup > Sharing Settings** and choose **Claim** under "Manage sharing settings for".
2. In Claim Sharing Rules click **New** and enter the label **Share Records based on Policy State CA**.
3. Set the criteria **Policy Account Holder State equals CA** and the access level **Read/Write**, then save.
4. Create a second rule for **TX** with the same access level.

[Insert screenshot: Claim sharing rules list]

Figure 7.6: Sharing rules defined on the Claim object

7.7 Permission Sets

Three Permission Sets separate the responsibilities of agents, adjusters and managers.

Table 7.4: Permission set matrix

Permission Set	Policy	Contact	Claim	System / Apex
Insurance Agent Access (quoting only)	Create, Read	Read, Edit	Read only	Flow User enabled; PremiumCalculator class
Claims Adjuster Access (claim handling)	Read only	–	Read, Edit	ClaimsAdjusterController class
Claims Manager Access (reporting and approvals)	Read only	Read, Edit	Read, Edit, Delete	Run Reports; Manage Approvals

1. Open **Setup > Permission Sets** and click **New**.
2. Enter the label (for example **Insurance Agent Access**); the API name is auto populated and save.
3. Open **Object Settings** and grant the permissions in Table 7.5, then open **System Permissions** and **Apex Class Access** to enable the items listed.
4. Repeat for **Claims Manager Access** and **Claims Adjuster Access**, then assign each set to the right users.

[Insert screenshot: Permission Sets page showing the three sets]

Figure 7.7: Permission Sets list

CHAPTER 8: TESTING AND VALIDATION

8.1 Testing Strategy

Testing combines declarative checks (running flows with sample data and reading the debug log), Apex unit tests and user acceptance checks that log in as each persona. Each milestone was tested immediately after its configuration, so that defects were found close to their cause.

8.2 Test Cases

Table 8.1: Functional test cases

ID	Scenario	Expected result	Status
TC-01	Create Auto policy with 17 character VIN	Record saved successfully	Pass
TC-02	Create Auto policy with 10 character VIN	Error: VIN must be exactly 17 characters	Pass
TC-03	Run AutoQuotingFlow with valid inputs	Draft policy created with Auto Record Type	Pass
TC-04	Quote for CA, model year 2015	Premium = 1250.00	Pass
TC-05	Quote for TX, model year 2020	Premium = 1050.00	Pass
TC-06	Create claim on Auto policy	Owner set to Auto queue	Pass
TC-07	Create claim on Property policy	Owner set to Property queue	Pass
TC-08	Create claim on Life policy	Owner set to Life queue	Pass
TC-09	Claim amount 60,000 with status New	Claim submitted to High Value Claim Approval	Pass
TC-10	Senior Adjuster approves, Manager approves	Approval Status = Approved	Pass
TC-11	Manager rejects at step 2	Approval Status = Rejected	Pass
TC-12	Open dashboard as adjuster	Only own claims shown as tiles	Pass
TC-13	Select Auto in dashboard filter	Only Auto claims shown without page reload	Pass
TC-14	Agent opens Claim object	Read only; cannot edit	Pass

8.3 Apex Unit Test Results

The test class creates its own data and does not rely on existing org records. A successful run shows the method `testGetAssignedClaims` as passed, and the coverage panel reports the coverage of `ClaimsAdjusterController`. Because the class contains one query, one wrapper constructor and one loop, a single well designed test exercising the full path covers the majority of lines; additional tests for a claim without a policy and for an adjuster with no claims raise the coverage towards 100 percent.

8.4 Observations and Known Limitations

- **Days Open calculation.** The wrapper computes `Date.today().daysBetween(dateOfLoss)`, which returns a negative number for past dates. The expression should be reversed (`dateOfLoss.daysBetween(Date.today())`) in a future release to display a positive count.
- **Hard coded queue Ids.** Routing uses fixed 18 digit Ids; a custom metadata type is a safer long-term option.
- **Policy state flow.** The state update flow loops over all policies; filtering the Get Records by the triggering claim's policy would be more efficient and precise.
- **Single test method.** Additional negative tests (no policy, no claims, null date of loss) would strengthen the suite.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Outcomes

All four milestones were implemented and verified. Agents can now create a draft Auto policy with an automatically calculated premium in a few clicks. Claims are routed automatically, adjusters see a personal dashboard, high value claims enter a controlled approval chain and every user sees only what their role allows.

9.2 Before and After Comparison

Table 9.1: Impact of the new system

Area	Legacy system	New Salesforce solution
Quoting	Manual and inconsistent	Guided Screen Flow with automatic premium
Data model	Separate per product line	One Policy object with Record Types and Field Sets
Data quality	Weak validation	Validation rule enforces VIN length
Claim assignment	Manual hand-off	Automatic routing by policy type
Adjuster workload view	Spreadsheets and email	LWC dashboard with filtering
Escalation	Follow-ups by phone or email	Automatic two-step approval
Security	Coarse access	Permission Sets and state based sharing
Customer view	Fragmented	360-degree view across Contact, Policy and Claim

9.3 Learning Outcomes

- Designing a flexible data model with Record Types and Field Sets.
- Building Screen Flows and Record-Triggered Flows for real business processes.
- Writing bulk-safe Apex, invocable methods and test classes with System.runAs.
- Developing parent and child Lightning Web Components with @wire and @api.
- Configuring approval processes, sharing rules and Permission Sets to implement least privilege.

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Multi-Line Insurance Policy and Claims Management System demonstrates how a legacy, manual insurance process can be replaced by one governed, automated Salesforce application. The combination of a flexible data model, guided quoting, Apex rating logic, automatic claim routing, an adjuster dashboard, multi-level approvals and role based security addresses each problem identified in the problem statement.

Equally important, the project shows how to balance declarative and programmatic development: Flows and configuration were used wherever possible for speed and maintainability, while Apex and LWC were reserved for rating logic and a high-performance user interface.

10.2 Future Enhancements

- Create Property and Life quoting flows reusing the same pattern and Field Sets.
- Replace the simulated database callout with a real named credential based integration for VIN decoding and property valuation.
- Introduce a rating matrix in custom metadata so that underwriters can change premium factors without editing code.
- Add routing based on adjuster level and claim history, and automatic escalation when a claim stays unresolved beyond a service level agreement.
- Build reports and a manager dashboard for loss ratio, claim cycle time and cross-sell opportunities.
- Add a customer portal (Experience Cloud) for policy view and online claim submission.
- Add Einstein or generative AI features for fraud detection and automated claim summaries.

REFERENCES

1. Salesforce Developers, Apex Developer Guide, developer.salesforce.com.
2. Salesforce Developers, Lightning Web Components Developer Guide, developer.salesforce.com.
3. Salesforce Help, Flow Builder and Record-Triggered Flows, help.salesforce.com.
4. Salesforce Help, Approval Processes, Sharing Rules and Permission Sets, help.salesforce.com.

5. Trailhead, Salesforce Developer learning paths, trailhead.salesforce.com.
6. Salesforce Lightning Design System (SLDS), lightningdesignsystem.com.
7. Project brief: Multi-Line Insurance Policy and Claims Management System (Naan Mudhalvan).